

4. Multimodal Visual Understanding Voice Interaction

4. Multimodal Visual Understanding Voice Interaction

1. Concept Introduction
 - 1.1 What is "Visual Understanding"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
 2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)
 - Code Analysis
- 3.1 Configuring Offline Large Model
 - 3.1.1 Configuring LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring Model Interface (`large_model_interface.yaml`)
- 3.2 Startup and Testing
4. Common Issues and Solutions
 - 4.1 Very Slow Response

1. Concept Introduction

1.1 What is "Visual Understanding"?

In the `largemodel` project, the **Multimodal Visual Understanding** feature enables the robot to not just "see" a pixel matrix, but to truly "understand" the content, objects, scenes, and relationships within an image. This is like giving the robot a pair of thinking eyes.

The core tool of this function is `seewhat`. When a user issues a command like "see what's here", the system invokes this tool, triggering a series of background operations, and ultimately feeds back the AI's analysis of the real-time scene to the user in natural language.

1.2 Brief Description of Implementation Principles

The basic principle is to input two different types of information -- **images (visual information)** and **text (linguistic information)** -- into a powerful multimodal large model (such as LLaVA).

1. **Image Encoding:** The model first converts the input image into digital vectors that the computer can understand through a Vision Encoder. These vectors capture the color, shape, texture, and other features of the image.
2. **Text Encoding:** At the same time, the user's question (such as "what's on the table?") is also converted into text vectors.
3. **Cross-modal Fusion:** The most critical step -- the model fuses the image vectors and text vectors in a special "Attention Layer". The model learns to "attend" to the parts of the image relevant to the question. For example, when asking about the "table", the model will focus more on regions in the image that match "table" features.
4. **Answer Generation:** Finally, a Large Language Model (LLM) component generates descriptive text as an answer based on the fused information.

Simply put, it **uses text to "highlight" the corresponding parts in the image, then uses language to describe the "highlighted" parts.**

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utis/tools_manager.py`)

The `seewhat` function in this file defines the execution flow of the tool.

```
# From largemodel/utis/tools_manager.py

class ToolsManager:
    # ...

    def seewhat(self):
        """
        Capture camera frame and analyze environment with AI model.
        """
        self.node.get_logger().info("Executing seewhat() tool")
        image_path = self.capture_frame()
        if image_path:
            # Use isolated context for image analysis.
            analysis_text = self._get_actual_scene_description(image_path)

            # Return structured data for the tool chain.
            return {
                "description": analysis_text,
                "image_path": image_path
            }
        else:
            # ... (Error handling)
            return None

    def _get_actual_scene_description(self, image_path, message_context=None):
        """
        Get AI-generated scene description for captured image.
        """
        try:
            # ... (Build Prompt)

            # Force use of a plain text system prompt with a clean, one-time
            context.
            simple_context = [{
                "role": "system",
                "content": "You are an image description assistant. ..."
            }]

            result = self.node.model_client.infer_with_image(image_path,
            scene_prompt, message=simple_context)
            # ... (Process result)
            return description
        except Exception as e:
            # ...
```

2. Model Interface Layer

(`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image understanding tasks, responsible for calling the specific model implementation based on configuration.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare messages)
        try:
            # Based on the value of self.llm_platform, determine which specific
            implementation to call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Tongyi model logic
                pass
            # ... (Other platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The implementation of this function involves two main layers: the tool layer defines business logic, and the model interface layer handles communication with the large language model. This layered design is key to achieving platform versatility.

1. Tool Layer (`tools_manager.py`):

- The `seewhat` function is the business core of the visual understanding feature. It encapsulates the complete workflow of the "look" action: first calling the `capture_frame` method to obtain an image, then calling `_get_actual_scene_description` to prepare a prompt for requesting the model to analyze the image.
- The most critical step is calling the model interface layer's `infer_with_image` method. It doesn't care which model is used underneath; it only needs to pass the two core pieces of data: "image" and "analysis instruction".
- Finally, it packages the analysis results received from the model interface layer (plain text descriptions) into a structured dictionary and returns it. This allows upper-level applications to conveniently use the analysis results.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function acts as a "dispatch center". Its main responsibility is to check the current platform configuration (`self.llm_platform`) and dispatch tasks to the corresponding processing functions based on the configuration value (such as `ollama_infer` or `tongyi_infer`).
- This layer is key to adapting to different AI platforms. All platform-specific operations (such as data encoding, API call formats, etc.) are encapsulated in their respective processing functions.

- In this way, the business logic code in `tools_manager.py` can support multiple different backend large model services without any modification. It only needs to interact with the unified, stable interface `infer_with_image`.

In summary, the execution flow of the `seewhat` tool embodies a clear separation of responsibilities pattern: `ToolsManager` is responsible for defining "what to do" (obtain image and request analysis), while `model_interface` is responsible for defining "how to do it" (select the appropriate model platform based on current configuration and complete the interaction). This makes the tutorial analysis universal, with consistent core code logic whether the user is in online or offline mode.

3.1 Configuring Offline Large Model

3.1.1 Configuring LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in terminal:

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

2. Modify/confirm `llm_platform`:

```
model_service:                                #Model server node parameters
  ros__parameters:
    language: 'zh'                            #Large model interface language
    useolinetts: True                         #This setting is invalid in text
mode, can be ignored

# Large model configuration
llm_platform: 'ollama'                       # Critical: Ensure this is set to
'ollama'
```

3.1.2 Configuring Model Interface (`large_model_interface.yaml`)

This file defines which visual model is used when the platform is set to `ollama`.

1. Open the file in terminal

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interface.
yaml
```

2. Find the ollama-related configuration

```
#.....
## Offline Large Language Models
# Ollama configuration
ollama_host: "http://localhost:11434" # ollama server address
ollama_model: "llava"                # Critical: Change this to your downloaded
multimodal model, such as "llava"
#.....
```

Note: Please ensure the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

3.2 Startup and Testing

1. **Start the `largemodel` main program:**

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

2. After successful initialization, say the wake word, then start asking: What do you see? Or describe the current environment.

3. **Observe results:**

In the first terminal running the main program, you will see log output showing the system receiving text commands, invoking the `seewhat` tool, and finally printing out the text description generated by the LLaVA model. Then the speaker will also broadcast the generated results.

4. Common Issues and Solutions

4.1 Very Slow Response

Issue: After asking a question, you need to wait a long time before getting a voice response.

Solution:

The inference cost of multimodal models is much higher than pure text models, so higher latency is normal.

1. **Use a smaller model:** In `large_model_interface.yaml`, try using a lighter `llava` model version.